Tig Cheat Sheet

Switch between views

m	Main.	p	Pager.	t	Directory tree.
d	Diff.	r	Refs.	f	File blob.
l	Log.	S	Status.	y	Stash.
B	Blame.	c	Stage.	h	Help.

Cursor Navigation

k / j	Move cursor one line up / down.
PgUp,-,a	Move cursor one page up.
PgDown, Space	Move cursor one page down.
Home / End	Jump to first / last line.

View Manipulation

q	Close current view, go to previous one until last.
Q	Quit tig.
Enter	Split the view to show the commit diff / In the diff view, scroll the view one line down.
Tab	Switch to next view.
R	Reload and refresh the current view.
O	Maximize the current view to fill the whole display.
Up	Move the cursor one line up. In diff view from the main view, point to the previous commit and update the diff view to display it.
Down	Similar to Up but will move down.
,	Move to parent.

Scrolling

Insert / Delete	Scroll view one line up / down.
w / s	Scroll view one page up / down.
Left / Right	Scroll view one column left / right.
|	Scroll view to the first column.

Searching

/	Search the view using RegExp.
?	Search backwards.
n / N	Find next / previous match.

View Specific Actions

u	Update status of file.
M	Resolve unmerged file by launching git-mergetool.
!	Checkout file (reset its content).
1	Stage single diff line.
@	Move to next chunk in the stage view.
] / [	Increase / Decrease the diff context.

Main View (m)

C	Cherry-pick the current commit (e.g. git cherry-pick %(commit)).

Diff View (d)

k / j	Move cursor one line up / down
Up / Down	Go to previous / next commit and update the diff view.

Blame View (B)

,	Load blame for the parent commit (Parent is queried for merges).
<	Return to blame for the child commit.

Refs View(r)

Enter	Open commit log view.
C	Checkout current branch (e.g. git checkout %(branch)).

Status View (S)

u	Add or stage file.
M	Resolve unmerged file by launching git-mergetool.
!	Checkout file (reset its content).
1	Stage single diff line.
@	Move to next chunk in the stage view.
] / [	Increase / Decrease the diff context.
C	Commit (e.g. git commit)

Stage View (c)

u	Stage only chunk for next commit (on a diff chunk), otherwise stage all changes.

Stash View (c)

A	Apply selected stash
P	Pop selected stash
!	Drop selected stash

Tree View (t)

,	Switch to the parent directory.

Prompt (:)

:<number>	Jump to the specific line number, e.g. :80.
:<sha>	Jump to a specific commit, e.g. :2f12bcc.
:<x>	Execute the corresponding key binding, e.g. :q.
:!<cmd>	Run a command, e.g. :!git log -p.
:<action>	Execute a Tig command, e.g. :edit.

Misc / Options

r	Redraw screen.
z	Stop all background loading (a repository with a long history without limiting the revision log)
v	Show version.
o	Open option menu
.	Toggle line numbers on/off.
D	Toggle date display on/off/short/relative/local.
A	Toggle author display on/off/abbreviated /email/email user name.
g	Toggle revision graph visualization on/off.
~	Toggle (line) graphics mode
F	Toggle reference (tag and branch) display on/off.
W	Toggle ignoring whitespace on/off for diffs
X	Toggle commit ID display on/off
%	Toggle file filtering in order to see the full diff instead of only the diff concerning the currently selected file.
$	Toggle highlighting of commit title overflow.
:	Open prompt to run a command.
e	Open file in editor.
G	Run garbage collection (e.g. git gc)

External Commands

Provide a way to easily execute a script or program. Bound to keys and use information from the current browsing state (e.g current commit ID).

Built-in external commands:	main	C	git cherry-pick %(commit)
	status	C	git commit
	generic	G	git gc

Set your custom bind Commands

In the .gitconfig
file, under the [tig "bind"] section, use pattern: keymap = key action

Keymaps	main, diff, log, help, pager, status, stage, tree, blob, blame, branch, and generic (global keymap)
Special Keys	Enter, Space, Backspace, Tab, Escape, Left, Right, Up, Down, Insert, Delete, Hash, Home, End, PageUp, PageDown, F1,..., F12

note: use ^ to set a keymap with the ctrl key (^j for ctrl-j)

External command control flags:

@	Run the command in background (no output).
?	Prompt the user before running the command.
<	Exit Tig after running the command.

Browsing state variables:

`%(head)`	Currently viewed head ID. Defaults to HEAD
`%(commit)`	Currently selected commit ID.
`%(blob)`	Currently selected blob ID.
`%(branch)`	Currently selected branch name.
`%(stash)`	Currently selected stash name.
`%(directory)`	Current directory path in the tree view; empty for the root directory.
`%(file)`	Currently selected file.
`%(ref)`	Reference given to blame or HEAD if undefined.
`%(revargs)`	Revision arguments passed on the command line.
`%(fileargs)`	File arguments passed on the command line.
`%(diffargs)`	Diff options passed on the command line.
`%(prompt)`	Prompt for the argument value.

ex:

```
[tig "bind"]
    # 'unbind' the default quit key binding
    main = Q none
    # Cherry-pick current commit onto current branch
    generic = C !git cherry-pick %(commit)
    # Amend last commit in status view
    #(prompting user and exit after)
    status = A !?<git commit --amend
```

For more informations on custom
external commands, see `http://jonas.nitro.dk/tig/tigrc.5.html`

---

tig project:`https://github.com/jonas/tig`
tig manual:`http://jonas.nitro.dk/tig/manual.html`